

BP3testconsoluciones.pdf



valks



Arquitectura de Computadores



2º Grado en Ingeniería Informática



Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación
Universidad de Granada



[Accede al documento original](#)

antes



**Descarga sin publi
con 1 coin**



Después

WUOLAH



Me fui de vacaciones'

Con 2.000€ que gané diseñando muebles

Un shot...



Diseñar

TUTROCITO

BP3

1. ¿Cuál de los siguientes mecanismos de interacción con el entorno en OpenMP no modifica directamente las variables de control internas?

- a) Variables de entorno
- b) Funciones del entorno de ejecución
- c) Cláusulas
- d) Variables de control internas (por definición)

2. ¿Qué variable de control interna está relacionada con el ajuste dinámico del número de threads y tiene un ámbito de "entorno de datos"?

- a) nthreads-var
- b) run-sched-var
- c) dyn-var
- d) def-sched-var

3. En OpenMP, ¿cuál es la variable de control interna que especifica el máximo número de threads para todo el programa y cómo se puede consultar o modificar?

- a) nthreads-var, consultable con función, modificable con variable de entorno o función
- b) thread-limit-var, consultable con función, modificable con variable de entorno
- c) thread-limit-var, consultable con función, no modificable
- d) def-sched-var, no consultable ni modificable

4. La variable de control interna def-sched-var afecta a la planificación por defecto de bucles. Según las fuentes, ¿cuál es su ámbito y cómo se puede consultar o modificar?

- a) Ámbito: entorno de datos; Consultar/Modificar: sí(f)/sí(ve,f)
- b) Ámbito: dispositivo; Consultar/Modificar: no/no
- c) Ámbito: el programa; Consultar/Modificar: sí(f)/sí(ve,f)
- d) Ámbito: entorno de datos; Consultar/Modificar: sí(f)/sí(ve,-)

5. ¿Cuál es el ámbito de aplicación de una variable de entorno de OpenMP (como OMP_NUM_THREADS) una vez modificada?

- a) La directiva OpenMP donde se usa
- b) La función C/C++ donde se llama a una rutina
- c) Los códigos que se ejecuten a partir de su modificación
- d) La región parallel o de trabajo compartida asociada

WUOLAH

6. Según el orden de precedencia para fijar el número de threads, ¿qué mecanismo tiene la máxima prioridad?

- a) El nº que fija la cláusula num_threads
- b) El contenido de la variable de entorno OMP_NUM_THREADS
- c) El nº que fija la función omp_set_num_threads()
- d) El nº que resulte de evaluar la cláusula if

7. Considerando el ejemplo de código if-clause.c, si el programa se ejecuta con un argumento de línea de comandos menor o igual a 4 ($n \leq 4$), ¿qué ocurrirá con la región marcada con #pragma omp parallel if(n>4)...?

- a) La región parallel se ejecutará en paralelo con el número de threads por defecto
- b) La condición if(n>4) no se cumplirá, por lo que no habrá ejecución paralela para esa región
- c) La cláusula if solo afecta si se usa la cláusula num_threads simultáneamente
- d) La cláusula if se ignora si nthreads-var o OMP_NUM_THREADS están definidos

8. En la cláusula schedule([modifier:]kind[,chunk]), ¿qué garantiza el modificador monotonic para un hilo (thread) que ejecuta las iteraciones de un bucle?

- a) Que las iteraciones se asignen en bloques de tamaño chunk
- b) Que si el hilo ejecutó la iteración i, después ejecutará iteraciones j tales que $j > i$
- c) Que no hay restricciones en el orden de asignación de los chunk
- d) Que la distribución de iteraciones se realiza en tiempo de ejecución

9. ¿Cuál de las siguientes afirmaciones sobre el tipo de planificación schedule(static) es correcta?

- a) La distribución se realiza en tiempo de ejecución y es apropiada si se desconoce el tiempo de cada iteración
- b) Las iteraciones se dividen en unidades de chunk y se asignan en round-robin
- c) Si no se especifica chunk, se usan unidades de una iteración
- d) Comienza asignando bloques largos que menguan gradualmente

10. ¿Qué comportamiento caracteriza a la planificación schedule(dynamic, chunk) si no se especifica un valor para chunk?

- a) Se asigna un único chunk a cada thread
- b) La granularidad de la distribución se fija por defecto según la implementación
- c) Se usan unidades de una iteración (chunk=1)
- d) La planificación pasa a ser equivalente a schedule(guided)

11. ¿Cuándo se recomienda utilizar los tipos de planificación `schedule(dynamic)` o `schedule(guided)`?

- a) Cuando el tiempo de ejecución de las iteraciones es desconocido
- b) Cuando la carga de trabajo por iteración es uniforme y predecible
- c) Cuando se necesita minimizar la sobrecarga de la planificación
- d) Cuando se desea que la distribución se realice en tiempo de compilación

12. En la planificación `schedule(guided, chunk)`, ¿cómo se determina el tamaño de los bloques de iteraciones que se asignan?

- a) Siempre es igual al valor de `chunk` especificado
- b) Es fijo y se asigna en round-robin a los threads
- c) Comienza con bloques grandes que menguan, calculado como las iteraciones restantes dividido por el número de threads, pero no más pequeño que `chunk` (excepto el último bloque)
- d) El tipo y tamaño del bloque se fijan en tiempo de ejecución mediante la variable de entorno `OMP_SCHEDULE`

13. La cláusula `schedule(runtime)` permite fijar el tipo de distribución en tiempo de ejecución. ¿De qué variable de control interna depende el tipo de distribución elegido en este caso?

- a) `dyn-var`
- b) `nthreads-var`
- c) `run-sched-var`
- d) `def-sched-var`

14. ¿Qué devuelve la función `omp_get_num_threads()` cuando se llama fuera de una región paralela activa?

- a) El número máximo de threads disponibles (`omp_get_max_threads()`)
- b) El número de procesadores disponibles (`omp_get_num_procs()`)
- c) 1
- d) Un valor indefinido, ya que solo es válido dentro de una región paralela

15. ¿Cuál es la diferencia clave en el resultado de `omp_get_thread_num()` y `omp_get_num_threads()` cuando se llaman dentro de una región paralela?

- a) Ambas devuelven el identificador del thread actual
- b) `omp_get_thread_num()` devuelve el identificador del thread actual, mientras que `omp_get_num_threads()` devuelve el total de threads en la región paralela activa
- c) `omp_get_thread_num()` devuelve el número total de threads, mientras que `omp_get_num_threads()` devuelve el identificador
- d) `omp_get_thread_num()` devuelve 0 para el thread master, y `omp_get_num_threads()` devuelve 1



ANA MENA



MYKE
TOWERS



DELA OSSA



MARÍA
BECERRA



RVFV



JC REYES



OMAR
MONTES

16. Según la clasificación de funciones de la biblioteca OpenMP en las fuentes, ¿qué categoría de funciones incluye `omp_get_wtime()` y `omp_get_wtick()`?

- a) Funciones para acceder al entorno de ejecución
- b) Funciones para usar sincronización con cerrojos
- c) Funciones para obtener tiempos de ejecución
- d) Funciones para interactuar con variables de control

17. La cláusula `if(scalar-exp)` en OpenMP puede usarse con varias directivas, pero la precaución mencionada en las fuentes es que solo se aplica a construcciones que incluyan:

- a) Bucles (DO/for)
- b) Regiones sections
- c) Regiones parallel
- d) Regiones single

18. Observando el ejemplo de cálculo de PI con OpenMP, ¿qué función se utiliza para obtener el tiempo de ejecución de la sección paralela?

- a) `clock_gettime()`
- b) `MPI_Wtime()`
- c) `omp_get_wtime()`
- d) `omp_get_wtick()`

19. ¿Qué diferencia fundamental se observa en el manejo de la suma total en el ejemplo de cálculo de PI con MPI, en contraste con el ejemplo de OpenMP?

- a) OpenMP usa una reducción explícita con `MPI_Reduce`, mientras que MPI usa una variable compartida
- b) MPI usa una variable local (`lsum`) para cada proceso y combina los resultados con `MPI_Reduce`, mientras que OpenMP usa una cláusula `reduction(+:sum)`
- c) Ambos usan la misma cláusula `reduction` para la suma
- d) MPI no calcula la suma total, solo la suma parcial por proceso

20. Según los datos de ejecución del ejemplo de cálculo de PI, ¿cuál de las tres implementaciones (C secuencial, OpenMP/C, MPI/C), muestra generalmente los tiempos de ejecución más bajos para 10,000,000 o 40,000,000 iteraciones en 8 threads/procesos en el computador descrito?

- a) C Secuencial
- b) OpenMP/C
- c) MPI/C
- d) Los tiempos son muy similares en las tres implementaciones para 8 threads/procesos

Respuestas:

1. c) Cláusulas
2. c) dyn-var
3. b) thread-limit-var, consultable con función, modificable con variable de entorno
4. b) Ámbito: dispositivo; Consultar/Modificar: no/no
5. c) Los códigos que se ejecuten a partir de su modificación
6. d) El nº que resulte de evaluar la cláusula if
7. b) La condición $\text{if}(n>4)$ no se cumplirá, por lo que no habrá ejecución paralela para esa región
8. b) Que si el hilo ejecutó la iteración i, después ejecutará iteraciones j tales que $j > i$
9. b) Las iteraciones se dividen en unidades de chunk y se asignan en round-robin
10. c) Se usan unidades de una iteración (chunk=1)
11. a) Cuando el tiempo de ejecución de las iteraciones es desconocido
12. c) Comienza con bloques grandes que menguan, calculado como las iteraciones restantes dividido por el número de threads, pero no más pequeño que chunk (excepto el último bloque)
13. c) run-sched-var
14. c) 1
15. b) `omp_get_thread_num()` devuelve el identificador del thread actual, mientras que `omp_get_num_threads()` devuelve el total de threads en la región paralela activa
16. c) Funciones para obtener tiempos de ejecución
17. c) Regiones parallel
18. c) `omp_get_wtime()`
19. b) MPI usa una variable local (lsum) para cada proceso y combina los resultados con `MPI_Reduce`, mientras que OpenMP usa una cláusula `reduction(+:sum)`
20. b) OpenMP/C